



Data Foundations

AICB-P1 · Ngày 7 · Embedding & Vector Store

Đặng Đức Huy

VinUniversity · Phase 1 · Tuần 1 · 2026

HÃY Suy Nghĩ...

“Agent trả lời sai vì model yếu, hay vì nó không có đúng dữ liệu để suy luận?”

Ví dụ: Agent CS dùng GPT-4 nhưng trả sai chính sách hoàn tiền — vì data là policy cũ 2023.”

Giữ câu hỏi này trong đầu khi học bài hôm nay



1. Data strategy cho AI product
2. Agent memory architecture
3. Embeddings
4. Vector store
5. Kết nối agent với data
6. Chunking & retrieval basics
7. Lab 7 + deliverable
8. Preview Day 08

- Hiểu **tầm quan trọng của dữ liệu** cho AI — data quality thường quyết định hơn model selection
- Phân biệt được **knowledge data, operational data, contextual data**
- Hiểu **embedding** là lớp biểu diễn nghĩa, không chỉ là một API call
- Hiểu được các bước và phương pháp **xử lý dữ liệu** trước khi đưa vào AI
- Hiểu **vector store** lưu gì, tìm gì, và lọc bằng metadata ra sao
- Mô tả được pipeline **Document → Chunk → Embed → Store → Query → Inject**
- Build được một **mini retrieval integration** để nối agent với dữ liệu riêng

Data inventory + chunking / embedding script + vector store index + semantic search demo + retrieval-enabled answer function

- 1 bộ dữ liệu mẫu đã được chunk và index
- 1 script truy vấn semantic search có trả kết quả liên quan
- 1 hàm trả lời sử dụng context retrieve được thay vì hỏi LLM “chay”

`src/` (code hoàn chỉnh) + `report/REPORT.md` (1 báo cáo / sinh viên)

- `src/chunking.py` — 3 chunking strategies + cosine similarity
- `src/store.py` — EmbeddingStore với search, filter, delete
- `src/agent.py` — KnowledgeBaseAgent (RAG pattern)
- 5 benchmark queries + so sánh kết quả giữa các strategy trong nhóm

01

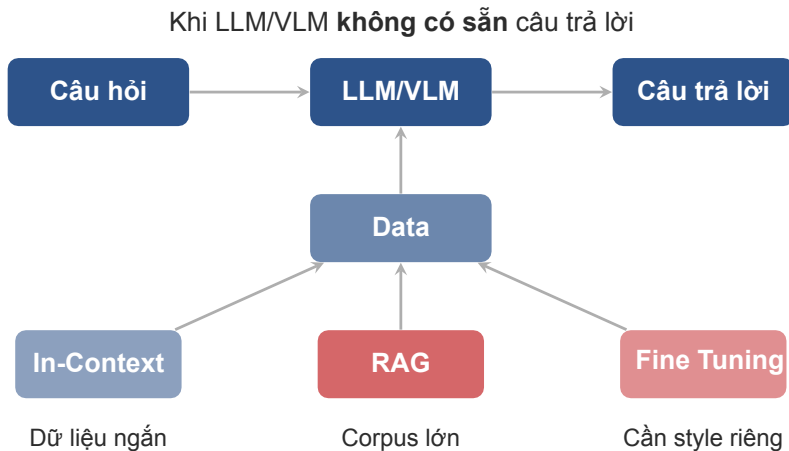
Data Strategy Cho Sản Phẩm AI

Không phải dữ liệu nào cũng nên đưa vào agent; chọn đúng dữ liệu quan trọng hơn nạp thật nhiều

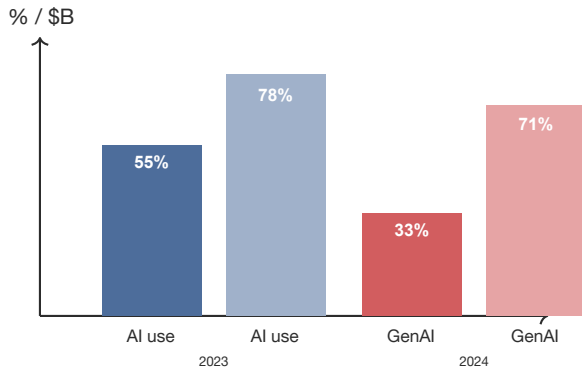
Khi LLM/VLM **biết** câu trả lời



Đơn giản — nhưng điều này hiếm khi đủ cho sản phẩm thật.



*Day 07 tập trung vào nhánh **RAG**: đưa đúng dữ liệu vào agent thay vì phải đổi model.*



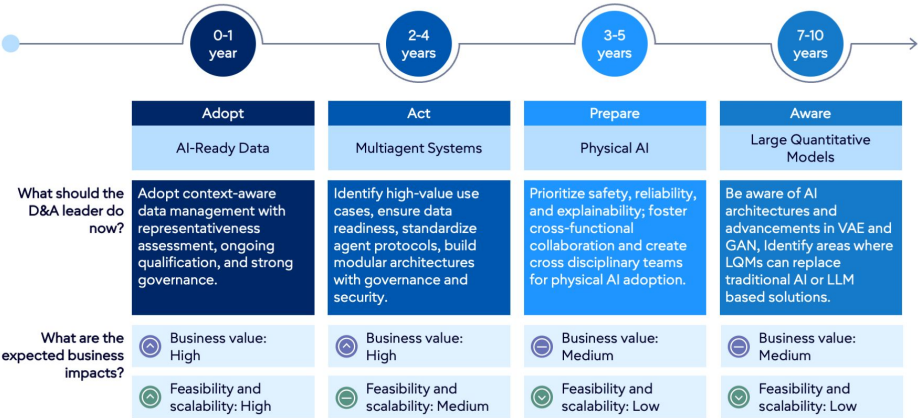
- AI adoption đã tăng rất nhanh.
- Câu hỏi chuyển từ “dùng model nào” sang “agent được phép biết gì”.
- Nếu dữ liệu sai, cũ, bẩn, hoặc không truy được đúng lúc, output sẽ yếu dù model mạnh.

Based on Stanford AI Index 2025.

Lưu ý: Data quality và retrieval quality thường quyết định trải nghiệm thật hơn là đổi sang model đắt hơn.

Emerging Data and Analytics Trends: Q1 2026

⬆ High ⬇ Medium ⬇ Low



Source: Gartner
850358

Dữ liệu bẩn / thiếu

- PDF scan lỗi OCR
- Policy cũ 2 năm chưa cập nhật
- Chunk cắt giữa câu
- Không có metadata

Kết quả: Agent hallucinate, trả lời sai, user mất niềm tin.

Dữ liệu sạch / đầy đủ

- Text đã chuẩn hóa, metadata đầy đủ
- Nguồn rõ ràng, có version
- Chunk theo section hợp lý
- Filter theo category + freshness

Kết quả: Retrieve đúng, answer grounded, có trích nguồn.

Lưu ý: Đổi model đắt hơn không sửa được data bẩn. **Hãy sửa data trước.**

Knowledge Data

Tài liệu, policy, SOP, FAQ, manual, hợp đồng, bài viết nội bộ.

Operational Data

Database, trạng thái đơn hàng, ticket, CRM records, logs, transactions.

Contextual Data

Session history, user profile, preferences, recent actions, channel context.

Knowledge data phù hợp với retrieval; **operational data** thường cần query có kiểm soát; **contextual data** nên được inject ngắn và đúng lúc.

Đặc điểm

- Ít thay đổi (static hoặc versioned)
- Dạng text dài: tài liệu, hướng dẫn, hợp đồng
- Cần chunk + embed để agent truy xuất
- Nguồn sự thật chính cho retrieval

Ví dụ thực tế

- FAQ / SOP nội bộ công ty
- Chính sách bảo hành, đổi trả
- Tài liệu kỹ thuật sản phẩm
- Legal contracts, compliance docs

Retrieval fit: Rất cao — đây là kiểu data lý tưởng cho vector store.

Đặc điểm

- **Thay đổi liên tục** (real-time hoặc near real-time)
- **Dạng structured:** SQL rows, JSON, logs
- **Thường truy xuất** bằng API / SQL, không embed
- **Cần access control** cực kỳ chặt chẽ

Ví dụ thực tế

- Trạng thái đơn hàng, tracking
- CRM records, ticket history
- Server logs, transaction records
- Inventory levels, pricing

Retrieval fit: Thấp — dùng function calling / SQL query thay vì semantic search.

Đặc điểm

- **Gắn liền với session / user** hiện tại
- **Thường ngắn gọn**, inject trực tiếp vào prompt
- **Có thể đến từ** profile store hoặc session state
- **Giúp agent** cá nhân hóa câu trả lời

Ví dụ thực tế

- **User profile:** tên, ngôn ngữ, vai trò
- **Lịch sử** hội thoại gần nhất
- **Giỏ hàng** hiện tại, trang đang xem
- **Kênh liên hệ** (chat, email, phone)

Retrieval fit: Trung bình — inject trực tiếp hoặc lookup đơn giản, ít khi cần semantic search.

Bối cảnh: Nhóm bạn đang build một **AI assistant** cho cửa hàng thời trang online. Khách hàng chat để hỏi về sản phẩm, đơn hàng, và đổi trả.

Nhiệm vụ thảo luận (10 phút):

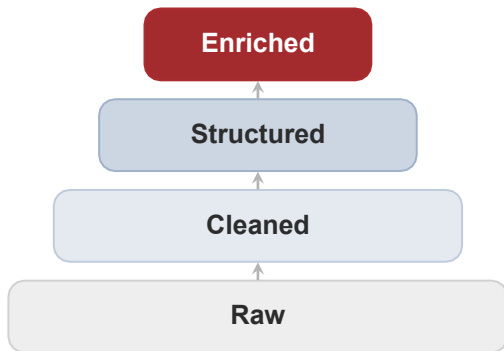
1. Liệt kê ít nhất **2 ví dụ cụ thể** cho mỗi loại data: **Knowledge** · **Operational** · **Contextual**
2. Với mỗi ví dụ, ghi rõ: **data đó nên được đưa vào agent bằng cách nào?** (*In-context? RAG? Query DB? Inject từ session?*)
3. Viết ra giấy hoặc note chung trong nhóm.

Trình bày & Phản biện:

1 nhóm lên chia sẻ (2 phút), các nhóm khác bổ sung hoặc phản biện.

Gợi ý thảo luận:

“Đơn hàng 12345 đang giao” thuộc loại nào? “Bảng size áo” thuộc loại nào? “User này hay mua size M” thuộc loại nào?



Tầng	Ví dụ cụ thể
Raw	PDF scan lộn, OCR ra “đổi trả”, HTML chứa tag rác
Cleaned	“đổi trả”, bỏ header/footer, chuẩn hóa Unicode
Structured	Chunk theo heading, gắn source refund-v3.pdf
Enriched	Tag category:support , access:public , quality:verified

Lưu ý: Sai lầm phổ biến: index ngay dữ liệu **raw** rồi kỳ vọng retrieval sẽ tự chữa mọi vấn đề.

Ví Dụ: Data Quality Ảnh Hưởng Retrieval Như Thế Nào?

Trước khi clean

Query: “thời hạn đổi trả”

Top-1 retrieved:

“...trang 3... header lập... \n\n mục lục... 30 ngày... xem thêm trang 7...”

Answer: **Hallucinate** vì context quá nhiều, agent lấy thông tin sai.

Sau khi clean + chunk

Query: “thời hạn đổi trả”

Top-1 retrieved:

“Khách hàng có 30 ngày kể từ ngày nhận hàng để yêu cầu đổi trả. Sản phẩm phải còn nguyên seal.”

Answer: Chính xác, grounded, có trích nguồn.

Cùng một model, cùng một query — chỉ khác data quality đã tạo ra kết quả hoàn toàn khác nhau.

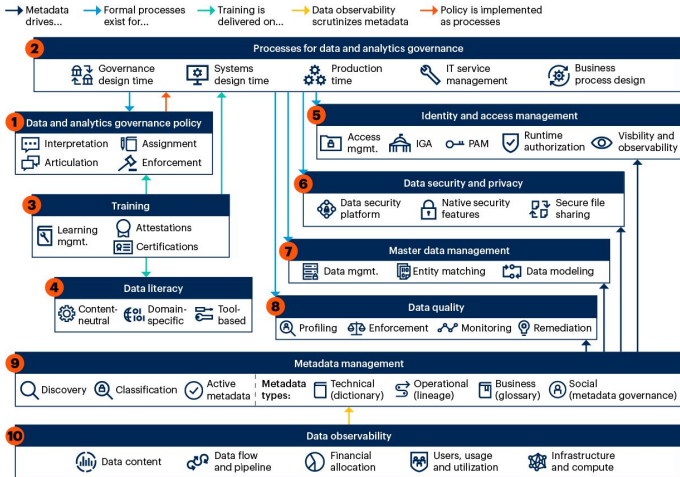
- ❑ **Ai sở hữu dữ liệu?** team nào chịu trách nhiệm cập nhật?
- ❑ **Ai được phép truy cập?** dữ liệu nào cần ACL, dữ liệu nào public nội bộ?
- ❑ **Data freshness** bao lâu phải re-index?
- ❑ **PII / sensitive fields** có cần mask trước khi embed không?
- ❑ **Không index mọi thứ theo kiểu “cứ nạp hết vào vector DB đã”.**

Ví dụ: FAQ do team CS sở hữu, public nội bộ, re-index mỗi tuần, không chứa PII.

LOẠI PII	VÍ DỤ	KỸ THUẬT MASK	MỨC RỦI RO
Tên cá nhân	"Nguyễn Văn A"	Thay bằng [PERSON]	Trung bình
Số điện thoại	"0912-xxx-xxx"	Regex replace	Cao
Email	"user@email.com"	Hash hoặc remove	Cao
CMND / CCCD	"012345678901"	Xóa hoàn toàn	Rất cao
Địa chỉ	"123 Lê Lợi, Q.1"	Generalize "Q.1, HCM"	Trung bình

QUY TẮC VÀNG: Mask trước khi embed — **không bao giờ** lưu raw PII trong vector store.

Reference Architecture: Data and Analytics Governance

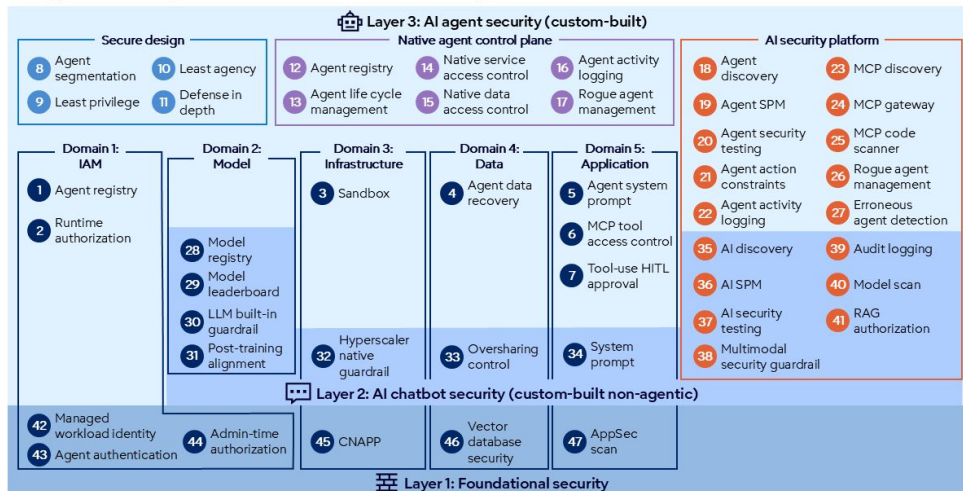


IGA = identity governance and administration; mgmt. = management; PAM = privileged access management

Source: Gartner

834531_C

AI Agent Security Architecture: A Detailed Analysis



Source: Gartner
841610

Agent Memory Architecture

02

Memory không chỉ là lưu nhiều hơn; nó là quyết định xem agent được nhớ gì, quên gì, và truy gì khi cần

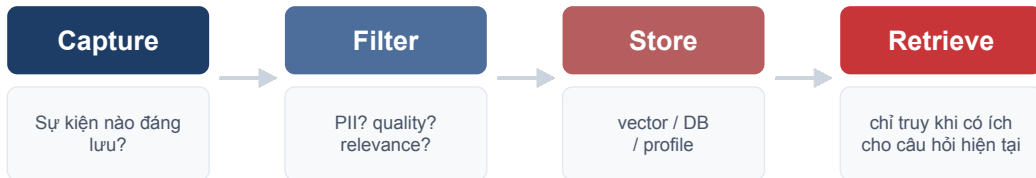
Short-term Memory

- Nằm trong context window
- Giữ lịch sử gần đây và task hiện tại
- Rẻ để dùng, nhưng dễ đầy token
- Phù hợp cho session logic ngắn

Long-term Memory

- Nằm ngoài context window
- Thường là vector store, DB, profile store
- Phải truy xuất khi cần
- Phù hợp cho tri thức tích lũy và user history chọn lọc

Context window không phải là vector store. Agent chỉ “nhớ dài hạn” khi có cơ chế truy xuất hoặc lưu trữ bên ngoài.



Lưu ý: Memory tốt là memory có **chính sách**. Lưu tất cả thường làm tăng chi phí, nhiễu, và rủi ro quyền riêng tư.

Những Nhầm Lẫn Thường Gặp

- Một prompt dài hơn không tự biến thành memory.
- Một file PDF upload một lần không phải memory nếu agent không truy được lại có chủ đích.
- Toàn bộ chat history không phải lúc nào cũng đáng inject.
- “Lưu cho chắc” thường tạo ra context nhiều hơn là context hữu ích.

Bản Chất Của Memory

Memory thực sự là sự kết hợp:

data + policy + retrieval

Nguyên lý cốt lõi: Thiếu một trong ba yếu tố (dữ liệu, chính sách, hoặc cơ chế truy xuất) thì hệ thống AI rất khó hoạt động ổn định và tối ưu.

System Prompt

~15%

Cố định mỗi request

Chứa hướng dẫn hệ thống, thiết lập persona và các guardrails bảo mật.

Retrieved Context

~35%

Chiếm nhiều token nhất

Các top-k chunks thông tin liên quan được inject trực tiếp vào prompt.

Conversation History

~25%

Cắt bớt khi quá dài

Lưu trữ các lượt hội thoại gần nhất để duy trì ngữ cảnh trao đổi.

Generation Budget

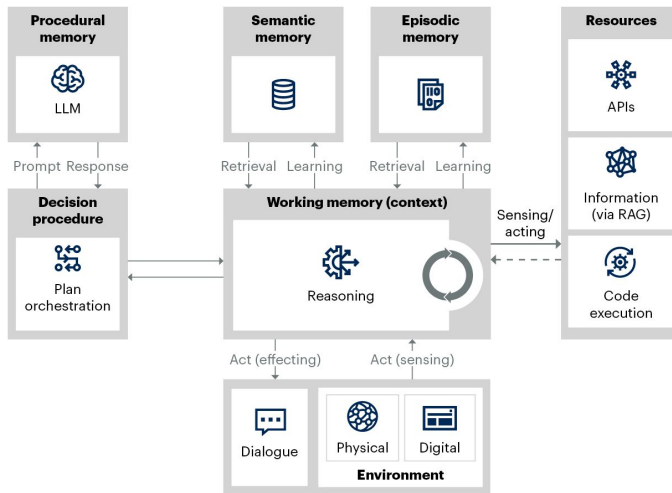
~25%

Token dành cho output

Nếu ngân sách này quá ít, câu trả lời của AI dễ bị cắt lửng giữa chừng.

Lưu ý: Nếu retrieved context chiếm quá nhiều, agent sẽ không đủ token để trả lời tốt. Luôn thực hiện **budget trước khi inject** nhằm tối ưu hóa không gian ngữ cảnh.

Types of Long-Term Agent Memory



Source: Gartner
823011_C

Data Processing

Xử lý dữ liệu để phục vụ cho AI rất quan trọng vì ảnh hưởng trực tiếp đến chất lượng câu trả lời

03

Format	Token hiệu quả	Giữ cấu trúc	Khi nào dùng
Markdown	★★★★★	Heading, list, table, bold	Mặc định được lựa chọn trong nhiều trường hợp
HTML (clean)	★★★	Bảng lồng, colspan, layout phức tạp	Khi MD mất thông tin cấu trúc
Plain Text	★★★ ★	Không có cấu trúc	Email, chat log, tran-script
JSON / YAML	★★★	Key-value, nested objects	Structured output, function calling

Nên chọn **Markdown**. So với HTML cùng nội dung, tiết kiệm ~30–50% token vì không có closing tags, attributes, class names. LLM được train trên lượng lớn MD nên hiểu rất tự nhiên.

BỐI CẢNH

Dịch chuyển sang HTML: Kỹ sư thuộc đội ngũ Claude Code (Anthropic) đưa ra quan điểm từ bỏ Markdown, chuyển sang sử dụng HTML cho toàn bộ tài liệu do AI sinh ra.

Bài toán chi phí: Các ý kiến phản đối lo ngại cấu trúc HTML tốn nhiều token hơn đáng kể (thẻ mở, đóng, class) làm tăng vọt chi phí vận hành API.

GIẢI PHÁP

Tối ưu hóa ~40% Token: Nicolas Bustamante (Founder Finom) xác nhận HTML tốn token hơn, nhưng đã tìm ra kỹ thuật tối ưu hóa hiệu quả cao.

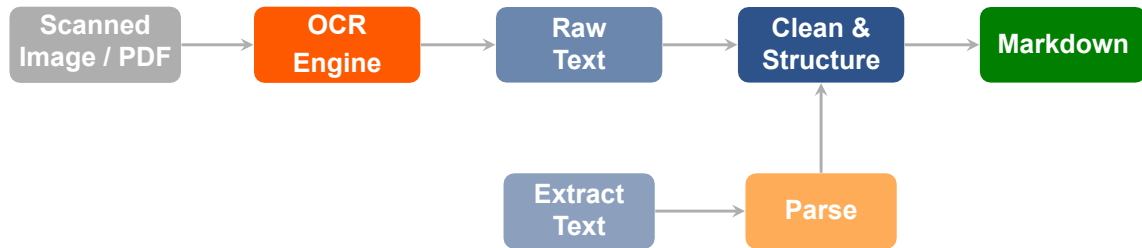
Kỹ thuật thực hiện: Áp dụng nguyên lý *Separation of Concerns*. Tách biệt hoàn toàn phần cấu trúc dữ liệu và phần hiển thị định dạng.

-44%

Giải Pháp Tách Biệt Định Dạng CSS

Quy trình tối ưu hóa chi tiết:

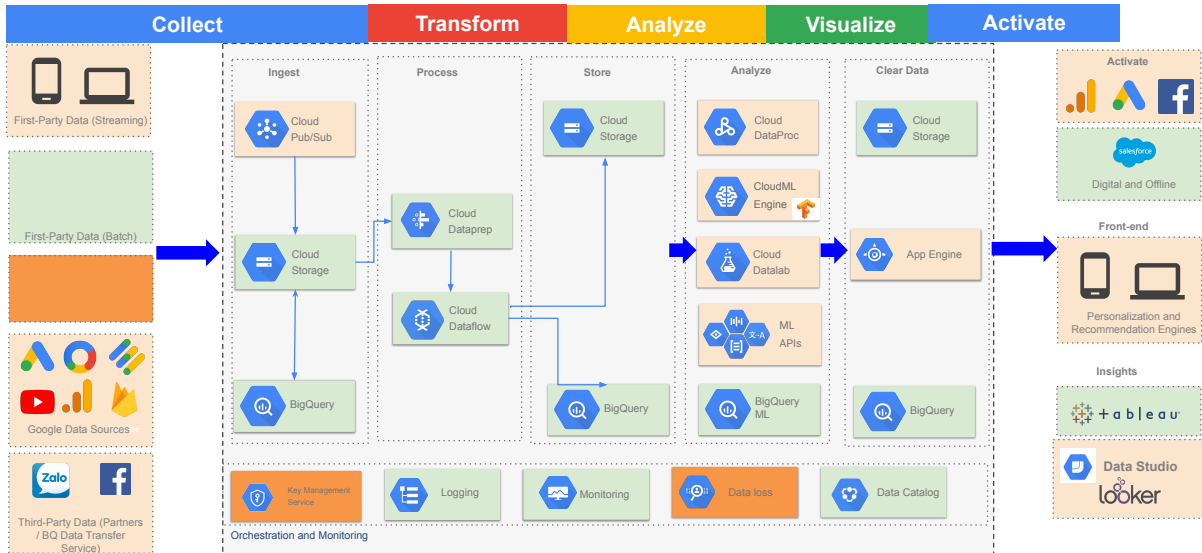
- **Bước 1:** Không để LLM tự viết khối định dạng CSS tốn kém tài nguyên trực tiếp trong thẻ.
- **Bước 2:** Thiết lập một file CSS tĩnh làm khung mẫu chung chuẩn hóa sẵn ở phía Client.
- **Bước 3:** Chỉ yêu cầu LLM sinh ra cấu trúc HTML thuần túy không có class hay thuộc tính style.
- **Kết quả:** Tiết kiệm tới **44% lượng token** tiêu thụ thực tế mà vẫn giữ nguyên khả năng hiển thị giàu cấu trúc.



*Đường thay thế cho tài liệu
có text sẵn (Word, HTML)*

Lưu ý: Sai lầm phổ biến: bỏ qua bước **Clean & Structure**, đưa raw text thẳng vào chunking. Kết quả: chunk chứa header/footer rác, OCR lỗi, ký tự đặc biệt — retrieval quality giảm rõ rệt.

Kiến trúc xử lý data thực tế



Embeddings

Embedding biến ngôn ngữ thành không gian toán học để máy có thể so sánh nghĩa thay vì chỉ so chữ

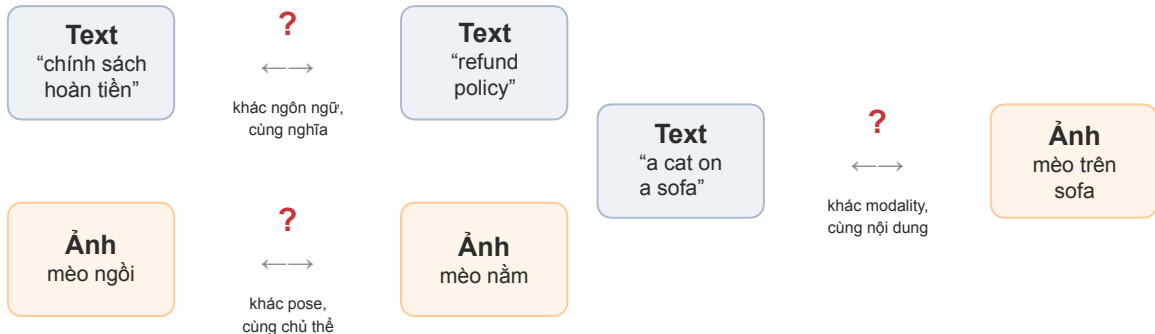
04

HÃY SUY NGHĨ...

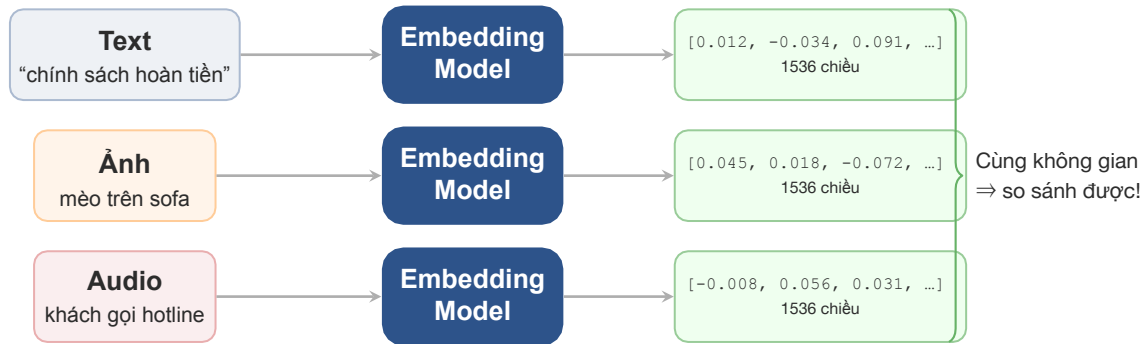
“Làm sao để máy biết hai thứ “giống nhau” — khi chúng không cùng ngôn ngữ, không cùng định dạng?”



Giữ câu hỏi này trong đầu khi học bài hôm nay



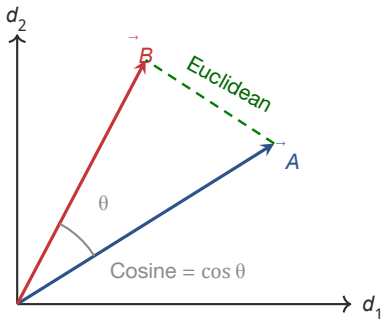
Con người nhìn là biết “giống nhau”. Nhưng máy cần biến text, ảnh, audio thành **số** trong cùng không gian — rồi mới đo khoảng cách được.



Embedding model là hàm biến dữ liệu thô (text, ảnh, audio) thành **vector số** cùng kích thước. Sau đó, đo khoảng cách giữa các vector = đo “độ giống nhau” về nghĩa.



- Text được biến thành vector nhiều chiều.
- Từ đó, máy có thể đo **độ gần về nghĩa**.
- Đây là nền tảng cho semantic search, clustering, dedup, recommendation.



Cosine similarity

- Đo **góc** giữa hai vector
- -1 đến 1; càng gần 1 = càng giống nghĩa
- Không bị ảnh hưởng bởi norm
- **Dùng nhiều nhất** trong NLP và retrieval

Euclidean distance

- Đo **khoảng cách thẳng** giữa hai điểm
- Càng nhỏ = càng gần
- Bị ảnh hưởng bởi scale / norm

Cosine là mặc định cho text embedding (so nghĩa, bỏ qua độ dài). **Euclidean** phù hợp khi cần đo khoảng cách tuyệt đối (image, geo).

Cosine Similarity

$$\cos(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \times \|\vec{B}\|}$$

- Tử: tích vô hướng (dot product)
- Mẫu: tích hai độ dài (chuẩn hóa)
- 1 = cùng hướng, 0 = vuông góc, -1 = ngược

Euclidean Distance

$$d(\vec{A}, \vec{B}) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

- Khoảng cách “đường chim bay” trong n chiều
- 0 = trùng nhau, càng lớn = càng xa
- 1536 chiều: công thức y hệt, chỉ nhiều i hơn

Lưu ý: Hầu hết vector store mặc định dùng **cosine**. Không cần tự code — nhưng cần hiểu score = 0.87 vs 0.31 nghĩa là gì.

Hiểu đơn giản về công thức Cosine Similarity

Hello, World! →

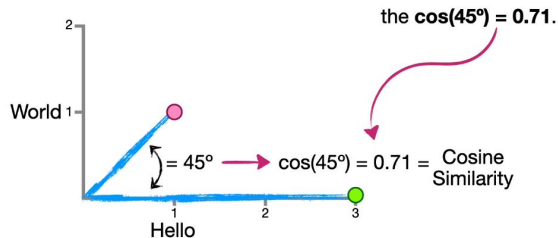
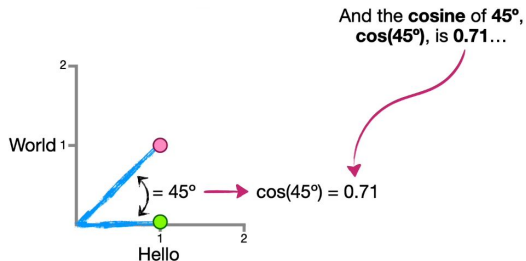
Hello	World
1	1
1	0

Hello! →

Hello, World! →

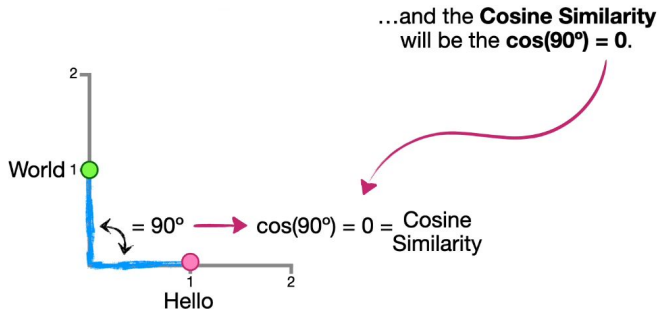
Hello	World
1	1
3	0

Hello Hello Hello! →

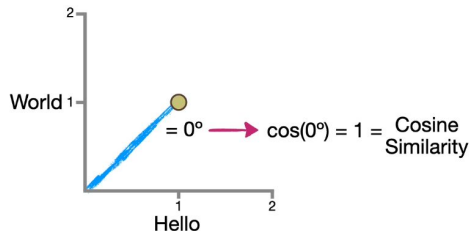


Hiểu đơn giản về công thức Cosine Similarity

	Hello	World
Hello →	1	0
World →	0	1



	Hello	World
Hello, World! →	1	1
Hello, World! →	1	1



Hiểu đơn giản về công thức Cosine Similarity

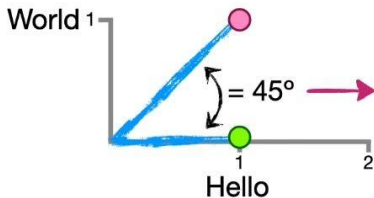
A = Hello, World! →

B = Hello! →

Hello	World
1	1
1	0

...which is the same value
we got when we took the
cos(45°).

$$\text{Cosine Similarity} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} = \frac{(1 \times 1) + (1 \times 0)}{\sqrt{1^2 + 1^2} \sqrt{1^2 + 0^2}} = 0.71$$



cos(45°) = 0.71 = Cosine Similarity

Hiểu đơn giản về công thức Euclidean Distance

1. Bản chất Hình Học & Tọa Độ

- Đo khoảng cách "đường chim bay" giữa 2 điểm p và q .
- Áp dụng định lý Pythagoras mở rộng cho không gian nhiều chiều.

dim	1	2
p	7.3	4.5
q	2.7	2.5
$ p - q $	4.6	1.9

$$\sqrt{4.6^2 + 1.9^2} = 5.0$$

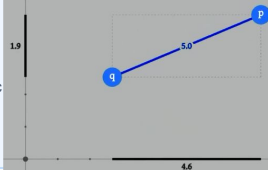
2. Công Thức & Cách Tính (2D)

Với 2 vector có tọa độ tương ứng:

$$d(p, q) = \sqrt{\sum (p_i - q_i)^2}$$

Thực quan hóa khoảng cách thực tế qua hiệu bình phương các chiều.

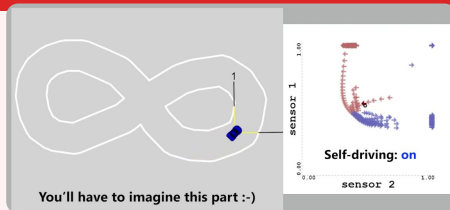
$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}$$



3. Ứng Dụng Thực Tế & Phân Tích Hệ Thống Tự Hành (Self-Driving)

Hệ thống ra quyết định dựa trên khoảng cách cảm biến:

- Cảm biến đo lường trạng thái môi trường dưới dạng vector đa chiều.
- Khoảng cách Euclidean nhỏ thể hiện sự tương đồng lớn về trạng thái vật lý thực tế.
- Giúp hệ thống phân loại chính xác các tình huống **Self-driving: ON / OFF** dựa trên ngưỡng an toàn.



Tính trên giấy hoặc máy tính (3 phút), post đáp án trên Discord:

Cặp 1

$$\text{vec}(A) = [1, 2, 3]$$

$$\text{vec}(B) = [2, 4, 6]$$

$$\cos(\text{vec}(A), \text{vec}(B)) = ?$$

Gợi ý: $\text{vec}(A) \cdot \text{vec}(B) = 1 \times 2 + 2 \times 4 + 3 \times 6$

Cặp 2

$$\text{vec}(C) = [1, 0, 0]$$

$$\text{vec}(D) = [0, 1, 0]$$

$$\cos(\text{vec}(C), \text{vec}(D)) = ?$$

Gợi ý: hai vector này có điểm chung nào không?

Lưu ý: Câu hỏi thêm: Cặp 1 có cosine = 1.0 mặc dù $\text{vec}(B) = 2 \times \text{vec}(A)$. Vì sao? Điều này nói gì về cosine similarity so với Euclidean distance?

Câu A	Câu B	Cosine	Nghĩa
“Chính sách hoàn tiền”	“Quy định đổi trả”	~0.87	Rất gần nghĩa
“Chính sách hoàn tiền”	“Cách giao hàng nhanh”	~0.52	Liên quan nhẹ
“Chính sách hoàn tiền”	“Thời tiết hôm nay”	~0.31	Không liên quan
“Chính sách hoàn tiền”	“Refund policy”	~0.82	Cross-lingual!

Embedding model hiểu nghĩa **cross-lingual**: “hoàn tiền” và “refund” gần nhau dù khác ngôn ngữ. Đây là sức mạnh so với keyword search.

Use case	Cách hoạt động	Ví dụ thực tế
Semantic search	Embed query + embed tài liệu, tìm cosine cao nhất	❑ User hỏi “trả hàng” → tìm được chunk “chính sách đổi trả” dù không trùng từ
Clustering	Gom các vector gần nhau thành nhóm	❑ 10,000 ticket CS → tự phân thành 15 chủ đề (hoàn tiền, lỗi app, giao hàng...)
Dedup	So cosine giữa các cặp, đánh dấu cặp > threshold	❑ Phát hiện 2 bài FAQ nội dung gần giống, gộp lại
Recommendation	Embed user intent + embed item, rank theo cosine	❑ User vừa đọc “React hooks” → gợi ý bài “useState patterns”

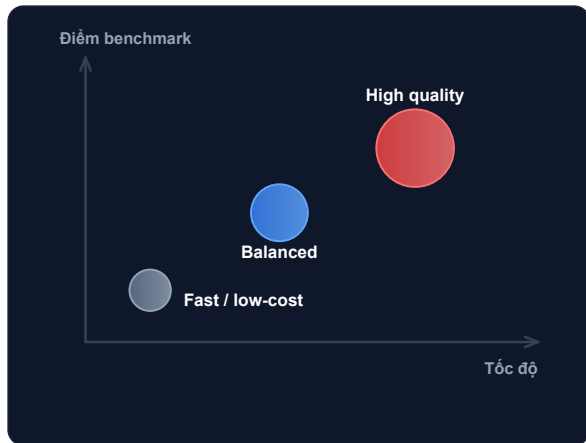
Lưu ý: Embedding không tự tạo ra “sự thật”. Nếu text nguồn sai, thiếu, hoặc chunk xấu thì retrieval vẫn tệ — **garbage in, garbage out**.

Model	Dim	MTEB Avg	Khi nào dùng
text-embedding-3-small	1536	62.3	Demo, PoC, lab (rẻ, nhanh)
text-embedding-3-large	3072	64.6	Production cần chất lượng
Open-source (e5, BGE)	768–4096	61–66	Self-host, data nhạy cảm, free

- Không có model “tốt nhất” cho mọi use case. Chọn theo **quality, latency, storage, language**.
- Với product thực tế, model cân bằng thường tốt hơn model mạnh nhất nhưng chậm và đắt.

Trên thực tế có hàng trăm embedding model (Cohere, Voyage, Gemini, Jina...). Xem bảng xếp hạng đầy đủ: MTEB Leaderboard trên Hugging Face.

Benchmark Framing: Quality, Speed, Size



Chiến lược lựa chọn

- **Không tối ưu tuyệt đối:** Không có model embedding nào là "tốt nhất" cho mọi trường hợp sử dụng thực tế.
- **Tiêu chí đánh giá:** Cần cân nhắc đồng thời giữa chất lượng (quality), độ trễ (latency), dung lượng lưu trữ (storage) và khả năng hỗ trợ ngôn ngữ (language coverage).
- **Tính cân bằng:** Trong môi trường production, một model có hiệu năng cân bằng thường tối ưu hơn là model mạnh nhất nhưng tốn tài nguyên và xử lý chậm.

Illustrative redraw based on the MTEB speed benchmark: quality, throughput, and embedding size are trade-offs.

Các Embedding Model Phổ Biến (2025–2026)

Phân tích đặc tính kỹ thuật & Kịch bản ứng dụng tối ưu

text-embedding-3-small

[OpenAI](#)

Dims: 1536 | Max: 8,191 tkn

Lựa chọn tối ưu về chi phí, tốc độ xử lý nhanh. Phù hợp làm baseline tốt cho mọi dự án.

text-embedding-3-large

[OpenAI](#)

Dims: 3072 | Max: 8,191 tkn

Tập trung nâng cao chất lượng biểu diễn không gian vector (Quality cao hơn bản small).

voyage-3

[Voyage AI](#)

Dims: 1024 | Max: 32,000 tkn

Thế mạnh đặc biệt trong việc xử lý Code và khả năng hỗ trợ đa ngôn ngữ (multilingual) mạnh mẽ.

Cohere embed-v4

[Cohere](#)

Dims: 1024 | Max: 512 tkn

Hỗ trợ đa ngôn ngữ xuất sắc, được tối ưu hóa đặc biệt cho các tác vụ tìm kiếm (search-tuned).

bge-m3

[BAAI \(Open-source\)](#)

Dims: 1024 | Max: 8,192 tkn

Mã nguồn mở (open-source), liên tục nằm trong nhóm dẫn đầu bảng xếp hạng MTEB (MTEB top tier).

◀ **Lời khuyên: Hãy chọn model theo use case thực tế của bạn, không nên chỉ nhìn vào điểm số MTEB.**

```
from openai import OpenAI
import numpy as np

client = OpenAI()
texts = ["Chính sách hoàn tín", "Quy định đi tr", "ỜThi tit hôm nay"]

resp = client.embeddings.create(
    model="text-embedding-3-small", input=texts
)
vecs = [np.array(d.embedding)for d in resp.data]

# Cosine similarity: ingha gn => score cao
cos = lambda a, b: a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
print(cos(vecs[0], vecs[1]))# ~0.87 hoàn tín <-> đi tr
print(cos(vecs[0], vecs[2]))# ~0.31 hoàn tín <-> Ờthi tit
```

Embedding Dưới Hood — Không Chỉ Là API Call

Tìm hiểu quy trình kỹ thuật chuyển đổi văn bản thành vector biểu diễn ngữ nghĩa

1 Input Text

"Chính sách đối
trà"

2 Tokenize

subword tokens

3 Transformer

self-attention

4 Pooling

mean / CLS

Output

[0.012, 0.87,
..]

1. Tokenizer & Phân rã văn bản

Chức năng chính: Chia văn bản đầu vào thành các đơn vị nhỏ hơn gọi là subword units sử dụng thuật toán **BPE** hoặc **SentencePiece**.

Ý nghĩa: Giúp mô hình xử lý được từ ngữ mới và giảm thiểu kích thước từ điển.

2. Transformer Layers & Ngữ cảnh

Cơ chế Self-Attention: Cho phép mỗi token được "nhìn" và tương tác với toàn bộ các token khác trong câu.

Ý nghĩa: Nắm bắt mối quan hệ ngữ pháp và ngữ nghĩa sâu sắc tùy thuộc vào ngữ cảnh xung quanh.

3. Pooling & Tổng hợp thông tin

Phương pháp: Gộp tất cả các biểu diễn vector của từng token thành một vector duy nhất đại diện cho cả câu (qua phương pháp **mean pooling** hoặc trích xuất **CLS token**).

4. Training với Contrastive Learning

Cơ chế tối ưu: Huấn luyện mô hình sao cho các câu có cùng ý nghĩa ngữ nghĩa sẽ được kéo lại gần nhau trong không gian vector, các câu khác nghĩa bị đẩy xa ra.

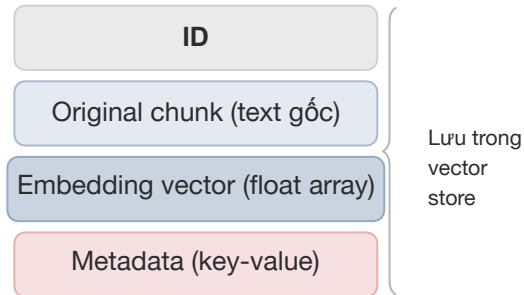
Lưu ý: Embedding không đơn thuần là ánh xạ từ điển, nó là cả một quá trình xử lý ngữ cảnh sâu sắc.

05

Vector Store

Vector store không chỉ lưu vector; nó lưu cả nguồn, metadata, và khả năng lọc để retrieval có ngữ cảnh đúng

Mỗi record gồm 4 thành phần:



Search results (top-k + scores)

Output khi query, không phải thứ được lưu

Ví dụ 1 record thật:

Trường	Giá trị
--------	---------

ID	policy-returns-001
Chunk	“Khách hàng có 30 ngày kể từ ngày nhận hàng để yêu cầu đổi trả. Sản phẩm phải còn nguyên tem.”
Vector	[0.012, -0.034, 0.091, ...] (1536 số)
Metadata	source: refund-v3.pdf category: support updated: 2026-03-01 access: public-internal

Vector dùng để *tìm* (semantic search).

Chunk dùng để *inject* vào prompt cho LLM.

Hai thứ khác nhau, lưu cùng nhau.

Trường metadata	Ví dụ	Tác dụng
Source / file name	refund-policy-v3.pdf	truy vết và hiển thị nguồn
Category	finance, support, legal	lọc đúng domain trước khi search
Time / freshness	2026-03-01	tránh dùng nội dung cũ
Access level	public-internal , restricted	giới hạn truy cập
Section / chunk id	returns.section.4	debug và cite chính xác hơn

Retrieval tốt cần semantic similarity kết hợp metadata filtering

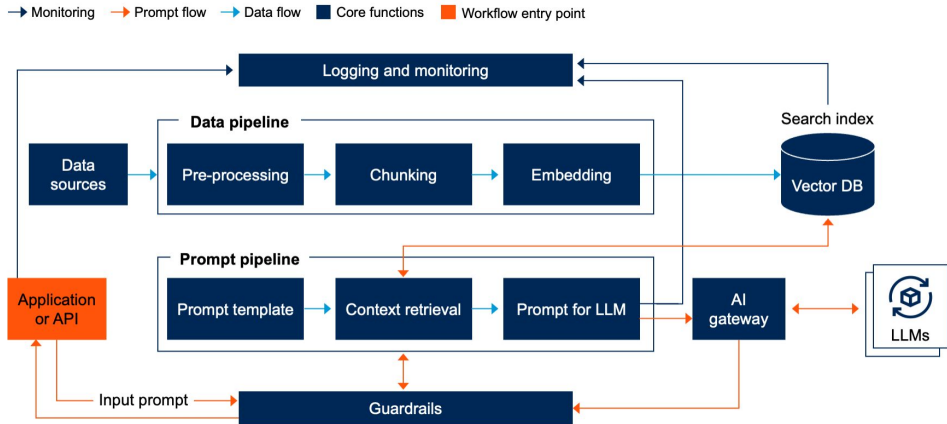
Tham số	Ý nghĩa	Ví dụ
Top-k	Lấy bao nhiêu chunk gần nhất	<code>n_results=3</code> đủ cho hầu hết câu hỏi đơn. <code>n_results=10</code> khi cần tổng hợp nhiều nguồn
Score threshold	Bỏ chunk có cosine thấp hơn ngưỡng	<code>Threshold = 0.7</code> : chỉ giữ chunk thật sự liên quan. <code>Threshold = 0.4</code> : chấp nhận liên quan nhẹ
Attribute filter	Lọc theo metadata trước khi search	<code>where={"category": "support"}</code> : chỉ tìm trong tài liệu CS, bỏ qua finance, legal

Filter theo category trước → semantic search → lấy top-3 có score > 0.6. Retrieval tốt thường đến từ **chunk đúng + metadata đủ + filter hợp lý** — không phải đổi model.

DB	Hosting	Language	Filtering	Scale	Best for
Chroma	Local	Python	Basic	Small	Prototype, học tập
Pinecone	Cloud	Any	Tốt	Large	Production SaaS
Weaviate	Hybrid	Any	Tốt	Large	Hybrid search
Qdrant	Hybrid	Rust	Rất tốt	Large	Filtering phức tạp
pgvector	DB addon	SQL	SQL where	Medium	Team đã dùng Postgres
FAISS	Local lib	C++/Py	Không	Any	Research, bench-mark

Chọn DB theo giai đoạn sản phẩm và yêu cầu vận hành

Modular Components in a RAG Architecture



Source: Gartner
837539

```
import chromadb

client = chromadb.Client()
col = client.create_collection("policies")

# 1. Add: chunk + metadata -> vector store
col.add(
    ids=["p1", "p2"],
    documents=["Khách hàng có 30 ngày đ i tr.", "Hoàn
               tin trong 7 ngày làmệvic."],
    metadatas=[{"cat": "returns"}, {"cat": "refund"}],
)

# 2. Query: semantic search
results = col.query(query_texts="đ i size", n_results=2)

# 3. Inject: dùng kt qu làm context cho LLM
context = "\n".join(results["documents"][0])
```

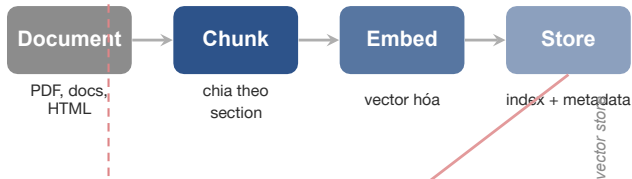
Kết Nối Agent Với Data

Retrieval pipeline là chiếc cầu nối giữa dữ liệu riêng và hành vi của agent

06

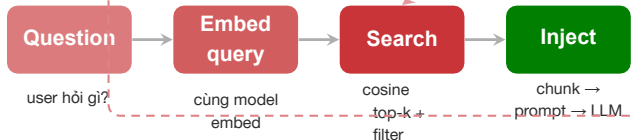
Ingestion

chạy 1 lần



Retrieval

mỗi câu hỏi

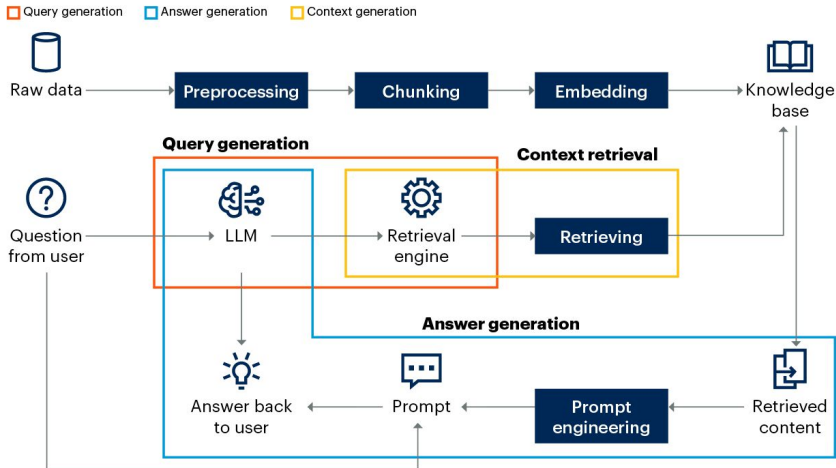


Lập 7 scope

Lưu ý: Ingestion chạy offline khi dữ liệu thay đổi. Retrieval chạy online mỗi khi user hỏi. Hai pha này tách biệt nhau.

Tham khảo Retrieval Pipeline theo Gartner

Retrieval-Augmented Generation (RAG) Pipeline



Source: Gartner
824147_C

1 file PDF / doc (nhiều section)

Chỉnh sách doi tra
Khách hàng có 30 ngày...

Chỉnh sách hoàn tiền
Hoàn tiền trong 7 ngày...

Giao hàng
Miễn phí đơn trên 500k...

↓ chunking

Chunk 1: chỉnh-sach-doi-tra (150 tokens)

Chunk 2: chỉnh-sach-hoan-tien (120 tokens)

Chunk 3: giao-hang (90 tokens)

Chunking = chia tài liệu dài thành các đoạn nhỏ hơn để embed và index riêng.

Vì sao phải chunk?

- Embedding model có giới hạn token input
- Một file dài embed thành 1 vector → không thể tìm *đoạn* liên quan, chỉ tìm *file* liên quan
- Chunk nhỏ → retrieval chính xác hơn, inject ít nhiễu hơn

Chiến lược chunk phổ biến:

- Theo **heading / section** (phổ biến nhất)
- Theo **số token cố định** (200–500 tokens)
- Theo **câu / paragraph**

Chunk quá to

Dính nhiều chủ đề vào cùng một vector, retrieve trùng nhưng inject rất nhiều.

> 1000 tokens

Chunk hợp lý

Một ý / một section trọn vẹn, có overlap 10 – 15% giữa các chunk liền kề.

200–500 tokens

Chunk quá nhỏ

Mất ngữ cảnh, retrieve nhiều mảnh rời rạc, khó tổng hợp thành câu trả lời tốt.

< 50 tokens

Khi cắt chunk, lấy thêm 1–2 câu cuối của chunk trước làm phần đầu chunk sau. Giúp giữ ngữ cảnh tại điểm cắt, tránh mất nghĩa giữa chừng.

Rule of thumb: bắt đầu đơn giản với chunk theo section / heading, rồi tối ưu sau bằng eval thay vì đoán cảm tính.

Tài Liệu Gốc (Chưa Xử Lý)

=== CHÍNH SÁCH ĐỔI TRẢ & HOÀN TIỀN ===

Hotline: 1900-xxxx | cs@shop.vn

- 1. Đổi trả:** Khách có 30 ngày từ ngày nhận hàng. Sản phẩm nguyên tem. Không áp dụng sale > 50%.
- 2. Hoàn tiền:** 7 ngày làm việc qua tài khoản/ví. Đơn COD hoàn tiền chậm hơn (10-14 ngày).
- 3. Giao hàng:** Miễn phí đơn > 500k. Nội thành 1-2 ngày, tỉnh 3-5 ngày. Không hỗ trợ ship quốc tế.

Cập nhật: Nguyễn Văn A, CS Team

Tài liệu nội bộ, không chia sẻ ngoài



Kết Quả Chunking Đề Xuất

Chiến lược: Chia theo Section/Chủ đề để tối ưu hóa ngữ cảnh tìm kiếm.

Chunk 1: Chính Sách Đổi Trả (~120 tokens)

Khách có 30 ngày đổi trả sản phẩm nguyên tem kể từ ngày nhận. Không áp dụng cho hàng sale > 50%. Liên hệ hotline 1900-xxxx.

Chunk 2: Chính Sách Hoàn Tiền (~110 tokens)

Hoàn tiền trong 7 ngày làm việc qua chuyển khoản hoặc ví điện tử. Lưu ý: các đơn hàng thanh toán COD sẽ hoàn tiền chậm hơn (10-14 ngày).

Chunk 3: Chính Sách Giao Hàng (~100 tokens)

Miễn phí giao hàng với đơn hàng > 500k. Thời gian giao nội thành từ 1-2 ngày, đi tỉnh từ 3-5 ngày. Chưa hỗ trợ vận chuyển quốc tế.

💡 Metadata đính kèm mỗi chunk:

Source: chinh-sach-cs-v3.1 | Category: FAQ | Date: 01/2026

Nhiệm vụ (10 phút):

- **1. Clean:** Bạn sẽ bỏ gì, giữ gì trước khi chunk? (header, footer, hotline?)
- **2. Chunk:** Chia tài liệu trên thành các chunks. Viết ra nội dung từng chunk.
- **3. Metadata:** Mỗi chunk cần gắn metadata gì? (source, category, date, ...)
- **4. Predict failure:** Viết 1 câu query mà strategy của bạn sẽ trả lời **sai** hoặc **thiếu**.

Trình bày: Mỗi nhóm chia sẻ strategy + predicted failure query (2 phút/nhóm).

Lưu ý: Câu hỏi thử thách: Nếu user hỏi "đổi size áo sale 60% được không?" — chunk nào sẽ được retrieve, và LLM sẽ trả lời đúng hay sai?

Chunk quá to

Q1: Hoàn tiền mất bao lâu? A1:
7 ngày.

Q2: Đổi size được không? A2:
Trong 30 ngày.

Q3: Ship quốc tế? A3:
Không hỗ trợ.

Query “đổi size” → retrieve cả 3 Q&A → LLM bị nhiễu bởi Q1, Q3.

Chunk hợp lý

Q2: Đổi size được không? A2:
Được, trong vòng 30 ngày kể
từ ngày mua.

Query “đổi size” → retrieve đúng Q2 → LLM trả lời chính xác.

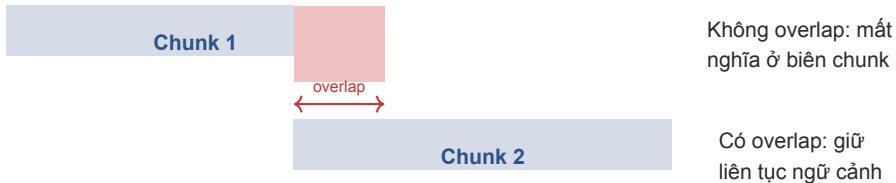
Chunk quá nhỏ

“Trong 30 ngày.”

Query “đổi size” → retrieve “Trong 30 ngày” → thiếu ngữ cảnh, LLM không biết 30 ngày cho việc gì.

Phương pháp	Cách hoạt động	Ưu điểm	Nhược điểm
Fixed-size token	Cắt mỗi N tokens	Đơn giản, dễ implement	Cắt giữa câu / ý
Sentence-based	Tách theo câu hoàn chỉnh	Giữ nghĩa câu	Câu dài / ngắn không đều
Section / heading	Tách theo heading (H1, H2)	Chunk trọn vẹn về ý	Cần tài liệu có structure
Recursive Semantic	Thử separator lớn → nhỏ Gom câu cùng chủ đề	Linh hoạt Chất lượng cao nhất	Phức tạp hơn Chậm, cần embed-ding

Không có “best” strategy — chọn theo đặc điểm tài liệu



- **Overlap thường 10–20%** kích thước chunk (e.g., 50–100 tokens cho chunk 512 tokens)
- Quá nhiều overlap → duplicate context, tốn storage, nhiều kết quả
- Quá ít overlap → mất nghĩa ở ranh giới giữa hai chunk

Retrieval

- Tìm context liên quan cho câu hỏi hiện tại
- Đọc từ knowledge base
- Trọng tâm: **relevance** và **grounding**

Ví dụ: User hỏi “chính sách đổi trả” → agent tìm trong FAQ → trả lời dựa trên tài liệu.

Memory

- Lưu trạng thái, sở thích, lịch sử chọn lọc
- Từ user profile hoặc interaction history
- Trọng tâm: **continuity** và **personalization**

Ví dụ: Cùng user hỏi lần 2 → agent nhớ lần trước đã đổi size M → gợi ý “bạn muốn đổi lại size khác?”

Lưu ý: Nhiều hệ thống dùng cả hai: **retrieval** để biết domain facts, **memory** để biết user này là ai và đã làm gì trước đó.

Query: “Chính sách đổi trả áp dụng trong bao lâu?”

Chunk xấu (raw, không section)

Retrieved (cosine: 0.61): “...giao hàng miễn phí đơn trên 500k. Đổi trả trong 30 ngày. Liên hệ hot-line 1900...”

LLM answer: “Bạn có thể đổi trả và liên hệ hotline 1900...” **(nhiều, thiếu chi tiết)**

Chunk tốt (theo section + metadata)

Retrieved (cosine: 0.89): “Chính sách đổi trả: Khách hàng có 30 ngày kể từ ngày nhận hàng để yêu cầu đổi trả. Sản phẩm phải còn nguyên tem.”

LLM answer: “30 ngày kể từ ngày nhận, sản phẩm còn nguyên tem.” **(chính xác, có nguồn)**

Lưu ý: Cùng model, cùng query — khác nhau hoàn toàn vì **chất lượng chunk**.


```
query = "Chính sách đổi trả áp dụng trong bao lâu?"
# = "Chính sách đổi trả áp dụng trong bao lâu?"
results = collection.query(query_texts=[query], n_results=3)

context = "\n".join(results["documents"][0])

prompt = f"""Dựa trên nguồn sau, hãy trả lời ngắn gọn. #
«Dựa trên nguồn sau, hãy trả lời ngắn gọn.»

Nguồn:
{context}

Câu hỏi: {query}
"""
```

LLM Chạy (No Retrieval)

- Chỉ dựa vào training data
- **Dễ hallucinate** với domain-specific
- Không cite được nguồn cụ thể
- Không cập nhật được data mới

RAG-Enhanced

- **Grounded** trên dữ liệu thật
- Giảm hallucination đáng kể
- Trích nguồn cho user verify
- Cập nhật bằng re-index, không retrain

RAG thêm **latency** (retrieve step) và **complexity** (pipeline), nhưng đổi lại **accuracy** và **trustworthiness** cao hơn nhiều.

Hosted / managed

File Search / managed retrieval giúp đi nhanh, giảm code hạ tầng, phù hợp demo và PoC.

Self-managed

Dùng Chroma / Faiss / vector DB khi cần kiểm soát chunking, metadata, pipeline, hoặc cost path chi tiết hơn.

Ngày 7 chỉ cần hiểu **trade-off**. Chưa cần sa sâu vào vector DB war hay tối ưu indexing nâng cao.

Pattern	Biểu hiện	Cách khắc phục
Chunk quá to	Retrieve trùng nhưng answer nhiều	Chunk nhỏ hơn, theo section
Thiếu metadata	Không filter được, lẫn do-main	Thêm category, source, date
Top-k quá cao	Inject quá nhiều context	Giảm k, thêm score threshold
Data cũ / stale	Trả lời đúng... nhưng outdated	Re-index định kỳ, version tracking
Query mơ hồ	Retrieve sai hoàn toàn	Query rewriting / expansion

Debug retrieval bắt đầu từ nhận diện pattern lỗi

Metrics quan trọng

- **Precision@k**: trong k kết quả, bao nhiêu thực sự relevant?
- **Recall@k**: trong tất cả docs relevant, bao nhiêu nằm trong top-k?
- **MRR** (Mean Reciprocal Rank): kết quả relevant đầu tiên ở vị trí nào?
- **NDCG**: thứ tự rank có tốt không?

Cách eval đơn giản cho lab

- Viết 10–20 test queries với expected docs
- Chạy retrieval, so sánh top-k vs expected
- Tính Precision@3 và Recall@3
- Ghi nhận failure cases: query nào retrieve sai?

Eval retrieval **trước khi** eval generation
— nếu retrieve sai thì generate không thể đúng.

Hands-on 7

Mục tiêu của lab là nối được dữ liệu riêng vào hệ thống AI thì
pipeline tối thiểu nhưng đúng bản chất

07

1. Chọn một bộ tài liệu nhỏ: FAQ, SOP, policy, internal docs
2. Chuẩn hóa text và quyết định metadata tối thiểu
3. Chunk dữ liệu theo section
4. Embed và lưu vào vector store
5. Viết semantic search demo
6. Inject top-k results vào answer function hoặc mini agent

Một knowledge base nhỏ nhưng truy đúng và trả lời grounded tốt có giá trị học tập hơn một dataset lớn nhưng nhiễu.

Yêu cầu tối thiểu

- 5–15 tài liệu (FAQ, SOP, policy)
- Mỗi tài liệu 1–5 trang
- Text đã extract sạch (không còn header/footer lộn)
- Ghi lại nguồn gốc mỗi file

Metadata tối thiểu cần ghi

- `source`: tên file gốc
- `category`: domain / loại tài liệu
- `date`: ngày tạo hoặc cập nhật
- `language`: `vi` / `en`


```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=64, separators=["\n\n",
    "\n", ". ", " "]
)

docs = load_documents("../data/") # your loader
chunks = []
for doc in docs:
    parts = splitter.split_text(doc["text"])
    for i, part in enumerate(parts):
        chunks.append({
            "id": f"{doc['source']}_chunk_{i}",
            "text": part,
            "metadata": {"source": doc["source"],
                "category": doc["category"]}
        })
```

```
import chromadb
from openai import OpenAI

client_ai = OpenAI() client_db
= chromadb.Client()

collection = client_db.create_collection("lab7_kb")

for chunk in chunks:
    resp = client_ai.embeddings.create(
        model="text-embedding-3-small",
        input=[chunk["text"]]
    )
    collection.add(
        ids=[chunk["id"]],
        embeddings=[resp.data[0].embedding],
        documents=[chunk["text"]],
        metadatas=[chunk["metadata"]]
    )
```

```
query = "thoi han doi tra san pham"
# = "thời hạn đổi trả sản phẩm"
results = collection.query(
    query_texts=[query],
    n_results=3,
    where={"category": "support"} # metadata filter
)

for i, doc in enumerate(results["documents"][0]):
    score = results["distances"][0][i]
    print(f"[{i+1}] score={score:.3f}")
    print(f"          {doc[:120]}...")
```

```
def answer_with_context(query, collection, model="gpt-4o-mini"):
    results = collection.query(query_texts=[query], n_results=3)
    context = "\n---\n".join(results["documents"][0])

    response = client_ai.chat.completions.create(
        model=model,
        messages=[{
            "role": "user",
            "content": f"""Dựa trên nguồn sau, trả lời ngắn gọn.
            Neu không tìm thấy, nói 'Không có thông tin'.
            # «Dựa trên nguồn sau, trả lời ngắn gọn. Nếu không tìm thấy, nói 'Không có thông tin'.»

            Nguồn:\n{context}\n\nCau hoi: {query}"""
        }]
    )
    return response.choices[0].message.content
```

Deliverable

- Data inventory 1 trang
- Script chunk + embed + index
- Query demo với 3 câu hỏi test
- Mini answer function có dùng retrieved context

Instructor check

- Query có retrieve đúng tài liệu không?
- Metadata có giúp lọc tốt hơn không?
- Câu trả lời có grounded hơn baseline không?
- Có biết chỉ ra failure case không?

- ☐ **Dữ liệu có owner và source rõ ràng**
- ☐ **Chunking có lý do, không cắt bừa theo số ký tự Metadata có ít nhất 2 trường hữu ích**
- ☐ **Có test query semantic, không chỉ test exact keyword**
- ☐ **Không chấp nhận câu trả lời nghe hay nhưng không chỉ ra được nguồn**

Data Inventory — Mẫu Template

Tài liệu	Loại	Owner	Freshness	PII?	Retrieval fit
FAQ v3.pdf	Knowledge	Support	Monthly	Không	Cao
Order status API	Operational	Eng	Real-time	Có	Thấp (API)
User profile	Contextual	Product	Per-session	Có	Inject trực tiếp
Legal contracts	Knowledge	Legal	Quarterly	Có	Cao (cần mask)

Template gợi ý — điều chỉnh theo domain thực tế

1. **“Nạp hết vào vector DB rồi tính sau”** — Garbage in, garbage out. Clean trước, index sau.
2. **Chunk cố định 500 chars cho mọi loại tài liệu** — Mỗi loại doc cần strategy khác nhau.
3. **Không có metadata** — Không filter được → retrieve lẫn domain → answer sai.
4. **Không eval retrieval** — Chỉ eval generation mà bỏ qua retrieval → debug mù.
5. **Embed raw PII** — Vi phạm privacy, rủi ro pháp lý, khó thu hồi.
6. **Top-k = 10 “cho chắc”** — Inject quá nhiều context → nhiều, tốn token, chậm.

- ❑ **Clean data trước khi embed** — bỏ header/footer lộn, chuẩn hóa encoding
- ❑ **Chunk theo structure** (heading, section) thay vì fixed-size
- ❑ **Thêm metadata** ít nhất: source, category, date
- ❑ **Overlap 10–20%** giữa các chunk liền kề
- ❑ **Eval retrieval riêng** trước khi eval end-to-end **Start small**: 10–20 docs, 3–5 test queries, iterate
- ❑ **Không fine-tune embedding model** trước khi đã tối ưu hết data + chunking + metadata

Cosine

Đo góc giữa 2 vector.
Không phụ thuộc độ dài.

Mặc định cho hầu hết
embedding models.

Euclidean (L2)

Khoảng cách “đường chim bay”.

Phụ thuộc magnitude.

Phù hợp khi vectors đã
normalize.

Dot Product

Nhanh nhất.

Kết hợp direction +
magnitude.

Tốt khi cần ưu tiên
“confidence”.

Với OpenAI embeddings (đã normalized), cosine $\approx$ dot product. Bắt đầu với **cosine**, chỉ đổi khi có lý do cụ thể.

Vấn đề: Brute-force search $O(n)$ quá chậm khi có hàng triệu vectors.

Giải pháp: ANN algorithms đánh đổi *một chút accuracy* để được *tốc độ gấp 100–1000x*.

Các thuật toán phổ biến:

- **HNSW** (Hierarchical Navigable Small World) — mặc định ở hầu hết DB
- **IVF** (Inverted File Index) — phân cụm rồi search trong cluster
- **PQ** (Product Quantization) — nén vector để tiết kiệm RAM

Ở quy mô lab ($<10K$ vectors), brute-force vẫn đủ nhanh. ANN chỉ cần khi scale lên production.

Lưu ý: Đây là kiến thức nền. Không cần implement ANN ở Day 7 — chỉ cần biết nó tồn tại.

Không chỉ text

- **Image embeddings:** CLIP, SigLIP
- **Audio embeddings:** CLAP, Whisper
- **Code embeddings:** Voyage Code, CodeBERT
- **Cross-modal:** tìm ảnh bằng text query và ngược lại

Ứng dụng thực tế

- E-commerce: search sản phẩm bằng ảnh
- Medical: tìm X-ray tương tự
- Code: semantic code search
- Creative: text-to-image retrieval

Day 7 focus vào text embedding. Multi-modal là hướng mở rộng sau.

Không nên dùng RAG khi:

- Câu hỏi hoàn toàn nằm trong training data (general knowledge)
- Data đã fit trong context window (đủ nhỏ để inject hết)
- Cần reasoning / tính toán, không cần factual lookup
- Task là creative writing, brainstorming

Nên dùng RAG khi:

- Cần domain-specific, proprietary knowledge
- Data thay đổi thường xuyên
- Cần cite nguồn, trích dẫn cụ thể
- Knowledge base quá lớn để fit trong context
- Cần kiểm soát access (ACL per doc)

Lưu ý: RAG không phải silver bullet. Nếu bài toán không cần external knowledge, RAG chỉ thêm latency và complexity.



Knowledge data: FAQ đổi trả, chính sách bảo hành, hướng dẫn sử dụng

Operational data: trạng thái đơn hàng (qua API)

Contextual data: lịch sử chat, user tier (VIP/regular)

Chunking: theo section heading

Metadata: category, language, version

Eval: 20 test queries, Precision@3 > 0.8

Fallback: “Tôi không tìm thấy thông tin, xin chuyển cho nhân viên.”

“Agent trả lời sai vì model yếu, hay vì nó không có đúng dữ liệu để suy luận?”

- Hầu hết trường hợp: **vì data**, không phải vì model.
- Data bẩn + chunk sai + không metadata → model mạnh nhất cũng hallucinate.
- Pipeline tốt (clean → chunk → embed → store → retrieve → inject) giải quyết 80% vấn đề.

Đầu tư vào **data quality** và **retrieval pipeline** trước khi nghĩ đến đổi model.

Phase 1 — Cá nhân

Hoàn thành TODO trong 3 file:

1. `src/chunking.py` — 3
chunking strategies + cosine
similarity
2. `src/store.py` — `EmbeddingStore`
`Document` và
(5 methods) `FixedSizeChunker` đã
3. `src/agent.py` — implement sẵn làm ví dụ.
`KnowledgeBaseAgent` (RAG) `Verify: pytest tests/ -v`

Phase 2 — Nhóm: So sánh Strategy

1. Nhóm chọn domain + thu thập
5–10 docs
2. Thống nhất 5 benchmark queries +
gold answers
3. Mỗi người thử strategy riêng
(chunking, metadata)
4. Chạy cùng queries, so sánh kết
quả
5. Demo + thảo luận liên nhóm

Cùng code, cùng model — khác **data strategy** sẽ cho kết quả rất khác. Data quality > model selection.

Hosted / managed

File Search / managed retrieval giúp đi nhanh, giảm code hạ tầng, phù hợp demo và PoC.

Self-managed

Dùng Chroma / Faiss / vector DB khi cần kiểm soát chunking, metadata, pipeline, hoặc cost path chi tiết hơn.

Hôm nay dùng **Chroma** (self-managed) để hiểu pipeline end-to-end. Hosted option tìm hiểu thêm ngoài giờ.

Nộp bài

- `src/` — hoàn thành tất cả TODO
- `report/REPORT.md` — 1 báo cáo / sinh viên

Phần	Điểm
Cá nhân (code + phân tích)	60
Nhóm (strategy + so sánh)	40
Tổng	100

5 góc nhìn tự đánh giá

1. **Retrieval Precision** — top-3 có đúng không?
2. **Chunk Coherence** — chunk giữ được ý trọn vẹn?
3. **Metadata Utility** — filter có giúp tăng chính xác?
4. **Grounding Quality** — answer dựa trên context?
5. **Data Strategy Impact** — strategy phù hợp domain?

- ❑ **Tests pass:** `pytest tests/ -v` không có FAIL
- ❑ **3 chunking strategies** đều implement và so sánh được **EmbeddingStore** search + filter + delete hoạt động **KnowledgeBaseAgent** trả lời dựa trên retrieved context
- ❑ **5 benchmark queries** có gold answers và kết quả so sánh
- ❑ **Report** giải thích approach + failure cases

Mỗi nhóm nghiên cứu **1 phương pháp chunking**, hiểu cách hoạt động, ưu/nhược điểm, và khi nào nên dùng. Sau đó trình bày cho cả lớp.

Phân công — mỗi nhóm chọn 1 chủ đề:

01

Recursive Character Splitting

LangChain-style, separator hierarchy

02

Semantic Chunking

Dùng embedding để tìm điểm cắt tự nhiên

03

Document-structure Chunking

Theo heading, table, list — Markdown/HTML aware

04

Agentic Chunking

Dùng LLM để quyết định chunk boundary

05

Parent-Child / Small-to-Big

Retrieve chunk nhỏ, inject chunk cha lớn hơn

06

Sliding Window + Overlap

Fixed-size với stride, trade-off coverage vs cost

Phase 1 — Nghiên cứu (20 phút):

- Tìm hiểu phương pháp: đọc docs, blog, paper, hoặc source code
- Chuẩn bị trình bày với 4 câu hỏi bắt buộc

Q1. Phương pháp hoạt động **như thế nào?** (cho người chưa biết)

Q2. Khi nào nên dùng, khi nào **không nên?** (ví dụ cụ thể)

Phase 2 — Trình bày (10 phút):

Các bạn xung phong chia sẻ kết quả, mỗi bạn trình bày **1 phương pháp chunking chưa được trình bày** (2 phút/bạn).

Tài liệu gợi ý:

LangChain docs (text splitters), LlamaIndex docs (node parsers), blog "Chunking Strategies for LLM Applications" (Pinecone).

01

Recursive Character

LangChain-style, separator hierarchy

02

Semantic Chunking

Dùng embedding tìm điểm cắt tự nhiên

03

Document-structure

Theo heading, table, list — Markdown aware

04

Agentic Chunking

Dùng LLM xác định ranh giới chunk

05

Parent-Child System

Retrieve chunk nhỏ, truy xuất ngữ cảnh cha

06

Sliding Window

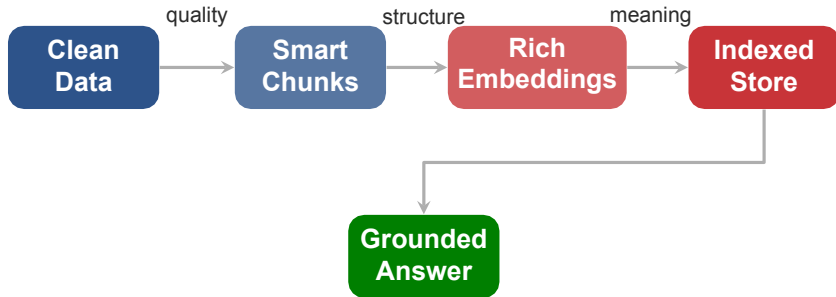
Kích thước cố định cùng độ gối overlap

Key Takeaways

Day 07 đặt nền cho RAG và mọi hệ thống AI dùng tri thức riêng

08

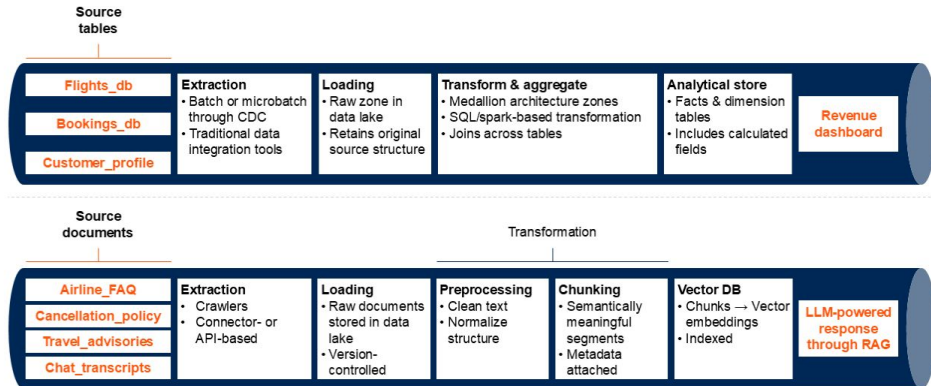
1. **Data quality** thường quan trọng hơn đối sang model đắt hơn.
2. **Embedding** là lớp dịch ngôn ngữ sang không gian có thể so sánh nghĩa.
3. **Vector store** là bộ nhớ dài hạn có thể tìm kiếm bằng ngữ nghĩa và metadata.
4. **Retrieval pipeline** là cầu nối từ dữ liệu riêng tới câu trả lời grounded của agent.



Lưu ý: Mỗi bước yếu sẽ kéo toàn bộ pipeline xuống. Hãy đầu tư đều vào từng bước thay vì chỉ tối ưu phần cuối.

How Data Engineers Can Enable RAG-Ready Data Pipelines

RAG Requires More Complex Data Pipelines



Source: Gartner
836573

1. Stanford HAI. *The 2025 AI Index Report*.
<https://hai.stanford.edu/ai-index/2025-ai-index-report>
2. OpenAI Docs. *Embeddings Guide*. <https://platform.openai.com/docs/guides/embeddings>
3. OpenAI Docs. *Retrieval Guide*. <https://platform.openai.com/docs/guides/retrieval>
4. OpenAI Docs. *File Search Guide*. <https://platform.openai.com/docs/guides/tools-file-search>
5. Hugging Face. *MTEB: Massive Text Embedding Benchmark*. <https://huggingface.co/blog/mteb>
6. Chroma Documentation. *Getting Started / Collections / Query*. <https://docs.trychroma.com/>
7. Karpukhin et al. *Dense Passage Retrieval for Open-Domain QA*. arXiv:2004.04906
8. Malkov & Yashunin. *Efficient and Robust ANN using HNSW Graphs*. arXiv:1603.09320

Bài tập sau buổi

- Hoàn thiện solution.py nếu chưa pass hết tests
- Rà lại knowledge base: bỏ nội dung nhiễu, thêm metadata
- Thử thêm 3 queries khó hơn, tìm failure cases mới

Preview Day 08

Ngày 8 đi tiếp sang **RAG pipeline**: indexing, retrieval, generation, evaluation.